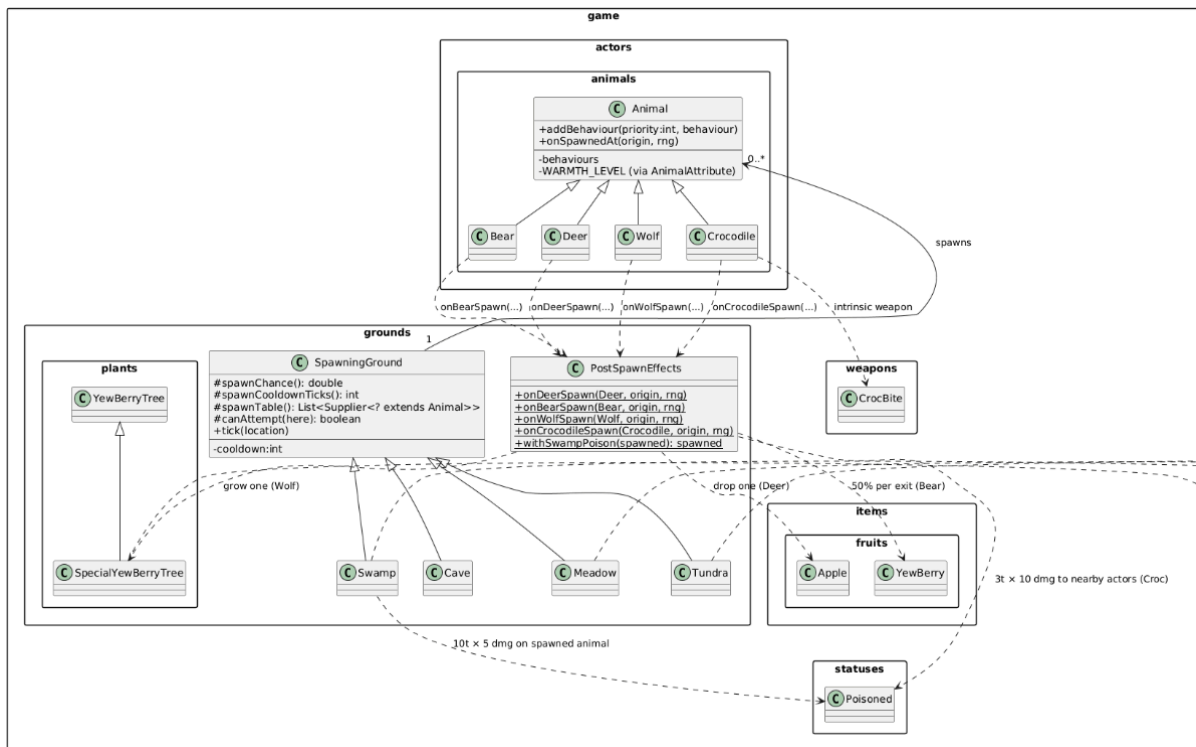


## A3 REQ 2 - Design Rationale

### UML Diagram



### Design 1

- SpawningGround (abstract)
  - Owns the attempt/cooldown loop, equal-chance spawnTable(), canAttempt(..) hook (used by Swamp), and post-placement hook call. No species knowledge lives here.
- Concrete spawners
  - Meadow: 50% every 7 ticks. Before returning suppliers, adapts animals to enable consumption (and attaches collection behavior where permitted by your policies) without instanceof scattered through the engine.
  - Cave: deterministic every 5 ticks.

- Tundra: 5% each tick, wraps suppliers to apply the +10 max HP and heal to new max.
- Swamp: only attempts when an adjacent actor exists; wraps suppliers to apply Poisoned(10×5) to the spawned animal.
- Animals
  - Animal (base): shared turn loop and behavior map; provides a small onSpawnedAt(Location, RNG) hook (default no-op).
  - Deer / Bear / Wolf / Crocodile: if they have spawn-side effects, they override onSpawnedAt and delegate to PostSpawnEffects helpers (no instanceof in the engine).
- PostSpawnEffects (utility)
  - onDeerSpawn(..), onBearSpawn(..), onWolfSpawn(..), onCrocodileSpawn(..) implement the cross-spawner rule set (apple drop, berry odds, special tree, neighbor poisoning).
  - Kept stateless and species-named for clarity; no central “apply()” with type switches.
- SpecialYewBerryTree
  - Standalone ground class (mature YewBerry tree) that drops a berry whenever a neighbor actor exists.
- Poisoned
  - Status ticking damage each turn; used by swamp-born poison (spawned animal) and croc-spawn neighbor poisoning.

Why this shape?

It cleanly separates where the thing spawns (spawner tiles) from what the thing does when it appears (animal-specific effect) and what environment grows (tree variant). We avoid leaking species logic into the engine core and keep supplier-based extensibility intact.

## **Principles Applied**

## DRY:

- The spawn control loop (cooldown, chance, neighbor selection) is centralized once in `SpawningGround.tick`.
- Tundra/Swamp use supplier-wrapping to inject effects without duplicating spawning code paths.
- Species post-spawn rules are written once (in `PostSpawnEffects`), re-used by all spawners.

## KISS

- **Equal-chance list of suppliers** keeps probabilities trivial (repeat entries for weights if needed).
- **Hooks over conditionals**: `canAttempt(..)` for Swamp, `onSpawnedAt(..)` for species-specific follow-ups. No big switch/if cascades.
- **ThreadLocalRandom**: removes the need to pass/store RNG state.

## SOLID

- **SRP**
  - `SpawningGround`: spawn timing/placement only.
  - Meadow/Tundra/Swamp/Cave: just configure/gate and decorate suppliers.
  - `PostSpawnEffects`: species-specific cross-spawner rules.
  - `SpecialYewBerryTree`: its own drop policy.
- **OCP**
  - New animals/spawners can be added by **adding classes** or **adding suppliers**; existing classes remain closed to modification.

- New spawn-side rules for a new animal? Implement `onSpawnedAt(..)` (and optionally add a helper in `PostSpawnEffects`).
- **LSP**
  - All spawners honor the `SpawningGround` contract; supplier returns still respect Actor semantics (can be placed, can enter ground, etc.).
- **ISP**
  - Consumers depend on small surfaces: spawners use `Supplier<? extends Actor>` and minimal engine APIs; species hooks expose one tiny method.
- **DIP**
  - High-level spawning depends on **abstractions** (suppliers & Actor) rather than concrete animal classes.
  - Species effects are invoked via **overridden hooks**, not by the engine checking concrete types.

## Connascence analysis:

- **Connascence of Name**
  - Shared method names like `spawnTable()`, `canAttempt(..)`, and `onSpawnedAt(..)` provide **uniform polymorphic dispatch**.
  - Capability constants (e.g., `Abilities.CAN_CONSUME`) rely on consistent naming to enable behaviors.
- **Connascence of Type**
  - `List<Supplier<? extends Actor>>` in the core spawner path constrains all spawnables to the **engine Actor type**.
  - Where we need animal-specific adaptation (e.g., Meadow attaching collection behavior), we **narrow to Animal** at the **edge** (constructor/setter), so the core engine remains Actor-generic.
- **Connascence of Meaning**
  - The **semantics** “Tundra ⇒ +10 max HP” and “Meadow ⇒ can consume ground items” are **codified** in the respective spawner wrappers. The same meaning is not duplicated elsewhere, keeping semantic consistency.

- **Connascence of Position**
  - Order-sensitive calls exist deliberately: in Tundra wrappers we must call **increase max HP** before **healing to max**; in the tick loop we must **choose dest** before **addActor**; these sequences are localized to avoid wide ripples.
- **Connascence of Execution (Timing)**
  - cooldown timing and “attempt only when neighbors exist” (Swamp) are **temporal dependencies** documented and encapsulated; post-spawn effects are invoked **only after** successful placement.
- **Connascence of Value**
  - Constants like “50% meadow”, “5% tundra”, “bear 50% per exit” and “croc 3×10” are **single-sourced** (no duplicates), easing audit/change.
- **Connascence of Algorithm (light)**
  - Equal-weight selection by supplier index is shared; if we change selection policy (e.g., to weighted), only one algorithmic locus changes.

## Trade-offs & maintainability foresight

- Supplier typing: Core remains `Supplier<? extends Actor>` for genericity; where we need animal-only adaptations (e.g., Meadow fruit-collection), we accept `Supplier<? extends Animal>` at that spawner’s boundary and upcast back to the core signature to integrate. This avoids instanceof in engine code, and keeps the unusual constraint localized.
- PostSpawnEffects utility vs. many tiny classes: we picked a clear, species-named static API to avoid a tangle of mini classes. If the rule set grows, we can pivot to a Visitor/registry without rewriting spawners.
- No stored Random: Using `ThreadLocalRandom` simplifies testing and avoids hidden dependencies. Where determinism is needed, `ForcedSpawners` (test doubles) control outcomes.

### Future examples

- New spawner “Volcano” that only attempts near lava and applies `Burning(10×5)`: implement `canAttempt(..)` and wrap suppliers.
- New animal “Boar” that scatters Hazelnuts in two random exits: implement `onSpawnedAt(..)` + add a `PostSpawnEffects.onBoarSpawn` helper.

### Alternative Design:

Design B — **Central Dispatcher** (one `apply(spawned, origin)` with a type registry)

## Pros

- One call site; easy to discover all rules.
- Could avoid modifying animal subclasses at all.

## Cons

- Tends toward **instanceof/type-switch** or a manual map from Class→Effect, concentrating species knowledge in one god-object.
- **Violates OCP**: every new species rule requires **editing** the dispatcher.
- Connascence of Type/Name balloons; harder to test in isolation.
- Encourages devs to put more non-spawner logic into the dispatcher (“just one more case…”).

Design C — **Event Bus / Observer** (**SpawnedEvent** published; subscribers react)

## Pros

- Very extensible; other systems can react to spawns (AI logs, analytics, quests).
- Loose coupling through events and handlers.

## Cons

- **Hidden control flow** (temporal connascence) → debugging who reacted becomes harder.
- Overkill for course scope; extra infra (bus, subscription life-cycle, ordering guarantees).
- Deterministic unit tests require **extra harness** to pump events and assert ordering.

## Conclusion

We selected Design A (Hooks + Supplier wrapping + tiny utilities) because it:

- Meets requirements cleanly: each rule maps to a single, obvious locus (spawner wrappers / onSpawnedAt helpers).
- Preserves engine genericity: SpawningGround knows only about Actor, not species.

- Minimizes connascence spread: temporal/semantic dependencies are localized to where they naturally belong (tundra HP change, meadow consumption, swamp gating/poison).
- Avoids instanceof in core loops, aligning with good OO design and your previous feedback.
- Keeps OCP strong: new animals/spawners add classes or suppliers, not edits to engine code.
- Tests well: deterministic “ForcedSpawners” and per-species helpers make unit tests focused and reliable.

In short, Design A gives you clarity, extension points, and predictable coupling—the best fit for the assignment’s evolving spawner rules and grading emphasis on DRY/KISS/SOLID and explicit connascence.